

Clock Skew Scheduling: Formulations, Algorithms, and Robust Extensions

A Survey Drawn from Course Lecture Notes on Algorithms for DFM and Network Optimization

Wai-Shing Luk

October 7, 2026

Abstract

Clock skew scheduling (CSS) is the problem of deliberately assigning the arrival times of clock signals at sequential elements so that every setup- and hold-time constraint is satisfied, while a secondary objective—the shortest feasible clock period, the largest timing slack, or the highest parametric yield—is optimized. This article presents a self-contained survey organized around one unifying idea: the timing constraints of a synchronous circuit form a system of difference constraints whose feasibility is equivalent to the absence of a negative cycle in a timing constraint graph (TCG). We develop the network-potential formulation, show that clock-period minimization, slack maximization, and yield-driven scheduling all reduce to parametric shortest-path problems (minimum mean cycle and minimum cost-to-time ratio cycle), and review the algorithmic toolkit—Bellman–Ford, Lawler’s binary search, Howard’s policy iteration, Karp’s minimum mean cycle, and the ellipsoid method for multi-parameter problems. We then extend the model with delay padding, multi-corner and multi-mode robustness, dual decomposition, and heavy-tailed (generalized extreme value) delay distributions, and close with clock-tree co-optimization and open problems.

1 Introduction

Synchronous digital design rests on a global clock that must reach every sequential element—every flip-flop (FF) and latch—at the right instant. Because the clock distribution network has finite, unequal delays, the active clock edge does not arrive at all registers simultaneously. The difference in arrival times is called *clock skew*. For a data path from register i to register j , with arrival times u_i and u_j , the (useful) skew is

$$y_{ij} = u_i - u_j.$$

Two facts make skew a first-class design variable. First, the clock is *periodic*, so a common shift $u_i \leftarrow u_i + c$ changes nothing: only the *differences* y_{ij} matter. Second, the setup- and hold-time constraints that guarantee correct data capture bound each skew from above and below, so each data path defines an interval of admissible skews. Clock skew scheduling (CSS) is the problem of choosing the arrival times—equivalently the skews—so that every timing constraint is met and some secondary objective is optimized.

Historically, designers targeted *zero skew* ($u_i = u_j$), which is easy to reason about: a single worst-case clock period can be verified by ordinary static timing analysis (STA). Zero skew, however, throws away a valuable degree of freedom. *Useful-skew* design [1], [2], [3] deliberately introduces nonzero skew to borrow slack from non-critical paths and lend it to critical ones. The result is either a higher clock frequency, a larger timing margin, or—under process variations—a higher parametric yield.

1.1 Why clock skew scheduling?

The value of a skew schedule is easily underestimated, because a schedule is a *plan* rather than a physical artifact and it is therefore tempting to dismiss it as an academic exercise. The practical considerations below, drawn from the author’s lecture [4], argue otherwise.

The flow order must respect skew scheduling. Scheduling requires accurate clock and data delays, which presuppose that global and clock-tree routing have already been performed. The conventional flow, however, treats clock-tree synthesis (CTS) as the most timing-critical step and performs it *before* routing the signal nets. A flow that is aware of useful skew would invert that priority: run CTS first, as if the signal wires were not yet fixed, and then re-route only the few nets whose geometry the clock tree displaces. This “CTS first, then re-route” ordering—rarely discussed—determines how much of the

available scheduling freedom can actually be realized.

Placement is the true lever. If the placement is poor, no algorithm inside CTS can balance the clock tree, because the imbalance is a property of the register locations, not of the clock tree. Placement must therefore anticipate the scheduler. The consequence is that useful-skew design is less a single algorithm than a *tool-chain contract*: placement, routing, and CTS must each be aware of, and cooperate with, the skew schedule.

A timetable, not a single departure time. The clearest intuition is a railway timetable. Trains do not all depart at the same instant, nor are they required to arrive simultaneously at every station; departures and arrivals are deliberately staggered to maximize the throughput of the network. Zero-skew clocking is the opposite policy—every register is clocked at the same instant—and it forgoes precisely the freedom that a timetable exploits. Useful skew is the timetable of the clock, and the asymmetry of current investment—large sums directed at lithography while the comparatively inexpensive software freedom of skew scheduling is left unexploited—becomes harder to justify as devices shrink and variation grows.

Why the formulation survives increasing complexity. The mathematical core of the problem is fixed at the outset and does not change as the model grows. The governing invariant is that only *relative* arrival times matter: shifting every clock by the same amount is physically meaningless, just as it is immaterial whether a schedule starts on Christmas or on New Year’s Day. This invariant is exactly the hypothesis under which the constraints form a system of difference constraints, so the negative-cycle feasibility test and the parametric machinery of Sections~4–8 survive the statistical, multi-corner, and multi-mode refinements almost unchanged. The scalar bisection of the single-parameter case generalizes to a Newton- or ellipsoid-style iteration when several parameters are present (Section~4.4); entirely new machinery is required only if a constraint is placed on *absolute* time, a situation that current practice does not produce. This stability is the strongest reason to study the problem in the abstract form developed below.

1.2 The central idea

This survey is organized around a single unifying observation:

The timing constraints of a synchronous circuit form a *system of difference constraints*,

$$u_i - u_j \leq w_e \quad \text{for every arc } e = (j, i),$$

and such a system is feasible if and only if the associated *timing constraint graph* (TCG) contains no *negative cycle*.

Consequently, the natural formulation places clock skew scheduling within *network optimization*. Let A be the node–edge incidence matrix of the TCG, u the vector of arrival times, and $y = Au$ the vector of skews. Then the timing problem is a *feasible potential problem*:

$$\underline{w} \leq y = Au \leq \overline{w},$$

and its optimization variants—minimum clock period, maximum slack, maximum yield—are all *parametric potential problems* of the form

$$\max\{\beta \in \mathbb{R} : y \leq d(\beta), \quad Au = y\},$$

where $d(\beta)$ is a monotone decreasing function of the scalar parameter β . When $d(\beta) = m - s\beta$ is affine with nonnegative denominators s , this is the *minimum cost-to-time ratio cycle* problem; when s is a constant it further reduces to the *minimum mean cycle* problem. Both admit fast combinatorial algorithms—Howard’s policy iteration, Karp’s characterization, and Lawler’s parametric search—and the negative cycle returned by these algorithms is not merely a certificate of infeasibility but also the *most critical* set of paths, which is exactly the information a designer needs.

1.3 Scope and organization

The remainder of the article is organized as follows. Section~2 fixes notation, derives the setup- and hold-time constraints, and constructs the timing constraint graph. Section~3 develops the network-potential formulation and the negative-cycle feasibility certificate. Section~4 treats clock-period minimization and slack maximization as parametric shortest-path problems, and extends them to multiple parameters via the ellipsoid method. Section~5 covers delay padding and its physical configurations. Section~6 addresses yield-driven scheduling under process variations, including the EVEN, PROP, and C-PROP methods. Section~7 extends the model to non-Gaussian, heavy-tailed delay distributions via the generalized extreme value (GEV) distribution. Section~8 discusses multi-corner and multi-mode robustness and dual decomposition. Section~9 compares the underlying algorithms and their complexities. Section~10 links the schedule to clock-tree synthesis, and Section~11 lists open problems. The presentation follows the author’s course lecture notes [4].

This article is a *survey*: it organizes and explains known formulations and algorithms rather than reporting new experiments, and the quantitative speedups quoted below are those reported in the cited literature. Where the abstract model diverges from physical design—clock-tree power and area, discrete cell sizes, spatial correlation, and hierarchical timing—the discrepancy is stated explicitly and revisited in Section~11.

2 Preliminaries: Timing Constraints and the Timing Constraint Graph

2.1 Local data paths and clock skew

Consider a data path that starts at the output of register i , passes through a block of combinational logic, and ends at the data input of register j (Figure~1). The data signal traverses the logic in a delay that ranges between a minimum d_{ij} and a maximum D_{ij} over the relevant operating conditions. The two registers are clocked by edges whose arrival times at i and j are u_i and u_j ; the skew seen by the path is $y_{ij} = u_i - u_j$.

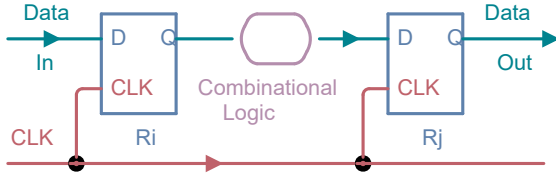


Figure 1: A local data path from register R_i to register R_j . The data traverses the combinational logic in a delay between d_{ij} and D_{ij} ; only the skew $y_{ij} = u_i - u_j$ of the two clock arrival times matters.

Figure~2 shows the corresponding clock waveforms: the arrival times are simply the delays of the clock edges at the two registers, and the skew is their difference.

The circuit as a whole is abstracted as a directed graph in which vertices are registers and edges are data paths. A small example, together with its timing constraint graph, is shown in Figure~3.

2.2 Setup- and hold-time constraints

Correct capture requires two inequalities. The *setup-time* constraint says that data launched by i must reach j early enough before the capturing edge:

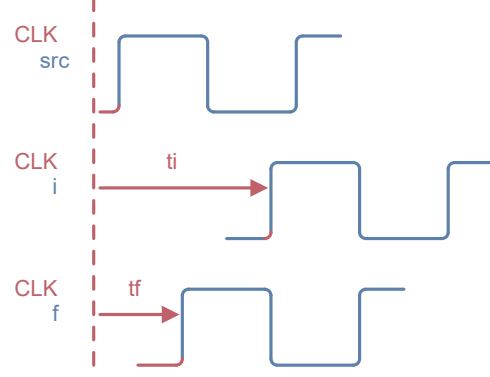


Figure 2: Clock waveforms. The source clock CLK_src and the clocks arriving at registers i and f are delayed by t_i and t_f ; their difference $t_i - t_f$ is the skew.

$$y_{ij} \leq T_{\text{CP}} - D_{ij} - T_{\text{setup}}. \quad (1)$$

Here T_{CP} is the clock period and T_{setup} the setup time of register j . A violation of (1) is a *cycle-time violation*, also called *zero clocking*. The *hold-time* constraint says that new data must not arrive at j so early that it races past the previous capture:

$$y_{ij} \geq T_{\text{hold}} - d_{ij}. \quad (2)$$

A violation of (2) is a *race condition*, also called *double clocking*. Combining the two, the admissible skew of path $i \rightarrow j$ lies in the *feasible skew region* (FSR)

$$\underline{w}_{ij} \leq y_{ij} \leq \bar{w}_{ij}, \quad \underline{w}_{ij} = T_{\text{hold}} - d_{ij}, \quad \bar{w}_{ij} = T_{\text{CP}} - D_{ij} - T_{\text{setup}}. \quad (3)$$

Because $y_{ij} = u_i - u_j$, the constraints (3) are exactly a system of *difference constraints* on the arrival times u .

2.3 The timing constraint graph

The FSR (3) is represented by a directed graph $G(V, E)$, the *timing constraint graph*, one vertex per register:

- each *setup* constraint contributes an *s*-edge from j to i with weight $\bar{w}_{ij} = T_{\text{CP}} - D_{ij} - T_{\text{setup}}$;
- each *hold* constraint contributes an *h*-edge from i to j with weight $-\underline{w}_{ij} = d_{ij} - T_{\text{hold}}$.

Indeed, the *s*-edge $j \rightarrow i$ encodes $u_i \leq u_j + \bar{w}_{ij}$, i.e., (1); the *h*-edge $i \rightarrow j$ encodes $u_j \leq u_i - \underline{w}_{ij}$, i.e., (2). A vertex assignment u satisfying every arc is precisely

a *feasible potential*, and we have the fundamental equivalence:

The timing constraints are satisfiable by some clock schedule if and only if the timing constraint graph has no cycle of negative total weight.

Summing the constraints around a cycle C eliminates the arrival times, because $\sum_{e \in C} (u_{\text{head}} - u_{\text{tail}}) = 0$; hence a cycle is feasible only if its weight is nonnegative. A negative cycle therefore certifies that *no* skew assignment can satisfy the corresponding constraints, and the only remedy is a circuit-level change—delay padding or logic restructuring (Section~5).

2.4 The zero-skew baseline and reported slacks

It is often convenient to start from a zero-skew analysis. If the clock is balanced so that $y_{ij} = 0$ for all paths, STA reports a *setup slack* S_{ij} and a *hold slack* H_{ij} for each path. Under useful skew, the constraints become

$$y_{ij} = u_i - u_j \leq S_{ij}, \quad -y_{ij} = u_j - u_i \leq H_{ij}.$$

Since $S_{ij} = T_{\text{CP}} - D_{ij} - T_{\text{setup}}$ is exactly the upper skew bound, the reported slacks are directly the FSR bounds relative to the zero-skew solution. This is why a modern STA engine that reports only slacks is already sufficient as a front end: the slacks *are* the edge weights of the TCG.

3 The Network-Potential Formulation

3.1 Flow, potential, and duality

The TCG is a *network* in the sense of discrete calculus. Its node–edge incidence matrix A maps node potentials to edge tensions, $y = Au$, while its transpose A^T is the boundary operator that maps edge flows to nodal divergences. A *flow* x satisfies conservation $A^T x = 0$ (a circulation); a *tension* y satisfies $\sum_{e \in C} y_e = 0$ around every cycle. Tellegen’s theorem, $x^T y = x^T A u = (A^T x)^T u = 0$, states that flow and tension are bi-orthogonal—the two faces of the same network (Figure~4).

3.2 The feasible potential problem

In this language the timing problem is the *feasible potential problem*

$$\text{find } u \quad \text{such that} \quad \underline{w} \leq y \leq \overline{w}, \quad Au = y. \quad (4)$$

Feasibility of (4) has a clean combinatorial characterization. Let $d^+(P) = \sum_{e \in P} d_e^+$ denote the *upper span* of a cycle P . Because $\tau^T y = \tau^T A u = 0$ for the indicator vector τ of any cycle, the constraints imply $d^-(P) \leq 0 \leq d^+(P)$ for every cycle P . Conversely, these cycle conditions are sufficient, and the “if” direction is constructive: it is exactly the shortest-path computation that produces u . In particular, when only upper bounds are present after adding reverse edges, the condition simplifies to $d^+(P) \geq 0$ for all cycles, and an infeasible instance returns a *negative cycle*.

3.3 Bellman–Ford and the case for lazy evaluation

The classical instrument for this task is the Bellman–Ford algorithm [5], [6] (Algorithm~1). It computes shortest-path potentials and, in a final pass, either certifies feasibility or exhibits a negative cycle. Its simplicity is appealing, but it has practical drawbacks that matter in timing closure:

1. it is designed to find shortest paths, with negative-cycle detection only a by-product;
2. it detects a negative cycle only at the *end* of the computation;
3. it requires all edge weights w to be known up front.

In an interactive timing-closure loop, the recommendation is therefore *lazy evaluation*: run the analysis on demand, vertex by vertex, and **stop as soon as a negative cycle is found**. The early-exit variant turns the algorithm from a global pass into a targeted repair tool.

4 Clock Skew Scheduling: Feasibility and Period Minimization

4.1 Minimizing the clock period

A traditional objective is to find the smallest clock period T_{CP} for which a feasible schedule exists. This is a linear program,

$$\begin{aligned} &\text{minimize} && T_{\text{CP}} \\ &\text{subject to} && \underline{w}_{ij} \leq u_i - u_j \leq T_{\text{CP}} - D_{ij} - T_{\text{setup}}, \quad \forall i \rightarrow j, \end{aligned} \quad (5)$$

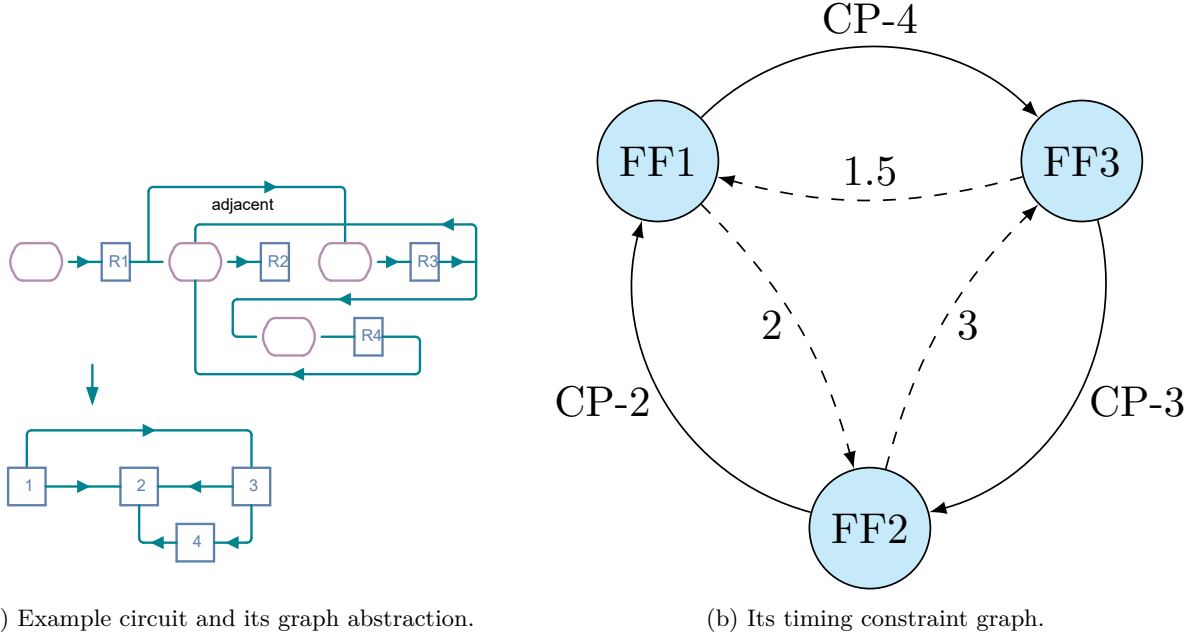


Figure 3: A circuit and its timing constraint graph. Solid arcs are setup constraints (s -edges); dashed arcs are hold constraints (h -edges). The weights are $T_{CP} - 4$ and 1.5, etc.

Algorithm 1: Negative-cycle feasibility test (Bellman–Ford).

Input: TCG $G = (V, E)$ with weights w_e ; a source r

Output: a feasible potential u , or a negative cycle

```

1 foreach  $v \in V$  do
2    $u_v \leftarrow 0$ ;  $\pi_v \leftarrow \text{nil}$ 
3 for  $k \leftarrow 1$  to  $|V|$  do
4   foreach  $(p, q) \in E$  do
5     if  $u_q > u_p + w_{pq}$  then
6        $u_q \leftarrow u_p + w_{pq}$ ;  $\pi_q \leftarrow p$ ;
7 foreach  $(p, q) \in E$  do
8   if  $u_q > u_p + w_{pq}$  then
9     return the negative cycle obtained by tracing  $\pi$ ;
10 return  $u$ 

```

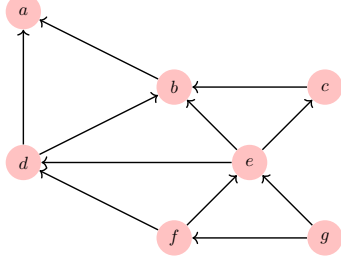


Figure 4: A flow (circulation) x and a tension (potential difference) y are dual objects on the same network.

whose constraints separate into a *hold part* independent of T_{CP} and a *setup part* that shrinks linearly as T_{CP} decreases. For a fixed T_{CP} , feasibility is decided by a single negative-cycle test on the TCG. Writing the cycle condition explicitly, a cycle C with k setup edges is nonnegative exactly when

$$T_{CP} \geq \frac{\sum_{e \in C^s} (D_e + T_{\text{setup}}) - \sum_{e \in C^h} (d_e - T_{\text{hold}})}{k}.$$

The smallest feasible period is therefore a *maximum cycle ratio*: it is obtained by maximizing a ratio of a “cost” sum to a “time” sum over cycles. This is the minimum-cost-to-time-ratio cycle problem, and when the denominator is constant across edges it becomes the minimum mean cycle problem.

4.2 The parametric potential problem

All of the objectives above are instances of one *parametric potential problem*:

$$\max\{\beta : y \leq d(\beta), Au = y\}, \quad d(\beta) \text{ monotone decreasing affine.} \quad (6)$$

If $d(\beta) = m - s\beta$ is affine with $s \geq 0$, (6) is the minimum cost-to-time ratio cycle problem; if s is a constant, it is the minimum mean cycle problem. If $d(\beta)$ is nonlinear, (6) still makes sense but the linear combinatorial machinery must be replaced by binary search or a convex solver. A useful sufficient condition is the following.

If $g(\beta)$ and every $f_e(\beta)$ are monotone decreasing, the parametric problem $\max\{g(\beta) : u_i - u_j \leq f_e(\beta)\}$ has a unique solution.

4.3 Cycle-based algorithms

Two families of algorithms solve (6) directly.

Binary search (Lawler) [7]. Maintain an interval $[\beta_{\min}, \beta_{\max}]$ bracketing the optimum. For a trial β , test the TCG for a negative cycle; if one exists, the trial is too optimistic and the upper bound moves down, otherwise the lower bound moves up. Each test costs one Bellman–Ford pass, and the number of tests is logarithmic in the inverse tolerance. Lawler’s method is simple and robust but can be slow.

Cycle cancellation (Howard) [8]. Howard’s policy iteration maintains a schedule (equivalently, a set of shortest-path trees) and repeatedly finds a cycle along which the ratio can be improved, “zeroing out” that cycle with the smallest possible effort. It converges very fast in practice and, as a by-product, returns the sequence of critical cycles in decreasing order of criticality. An improved variant introduces binary-search steps to accelerate convergence; hybrid methods combine the strengths of both.

4.4 Multi-parameter problems and the ellipsoid method

Every formulation so far optimizes a *single* scalar parameter—the clock period, a uniform slack, or one yield figure—and is therefore solvable by the combinatorial algorithms above. A real design, however, is often governed by *several* parameters at once: the clock period together with the timing slack, several yield targets, or the periods of multiple clock domains. The general problem is

$$\max\{g(\beta) : t_i - t_j \leq f_{ij}(\beta) \ \forall (i, j) \in E\}, \quad \beta \in \mathbb{R}^p, \quad (7)$$

where g and the f_{ij} are monotone in each component of β (and, for the convex extension, convex) but need not be affine. For $p = 1$ this is the parametric potential problem (6), solved by the minimum-ratio-cycle algorithms. For $p > 1$, parametric shortest-path theory no longer applies, and one must turn to a general convex optimization method.

The reason the problem stays tractable is that a *negative-cycle test is a separation oracle*. Given a trial β , either the timing constraint graph $G(\beta)$ is feasible, or it contains a negative cycle C ; in the latter case

$$\sum_{e \in C} f_e(\beta) < 0$$

is a violated constraint that separates β from the feasible set. An oracle therefore needs only *one* vio-

lated constraint per iteration and never requires all $|E|$ constraints—indeed, all path delays—to be known in advance. This is the “lazy evaluation” principle of Section 3 lifted from feasibility to optimization.

The *ellipsoid method* [9], [10] is the natural engine for such an oracle: it maintains an ellipsoid known to contain the optimum, and each oracle call supplies a separating half-space through the current center, after which the ellipsoid is shrunk to the appropriate side until its volume (and hence the optimality gap) drops below a tolerance. Because it needs only separation, it tolerates non-differentiable objectives and nonlinear constraints; conceptually it is the multi-dimensional generalization of the bisection strategy used by Lawler’s algorithm. Its update is

$$\beta_{k+1} = \beta_k - \frac{1}{p+1} \frac{A_k g_k}{\sqrt{g_k^\top A_k g_k}},$$

where A_k defines the current ellipsoid and g_k is the separating gradient; deep, central, and parallel-cut refinements accelerate convergence in practice.

This is exactly the architecture implemented by the `netoptim` package, whose `NetworkOracle` wraps Howard’s negative-cycle finder and, on detecting a negative cycle, returns a cutting plane—its gradient is the negative sum of the edge sub-gradients and its intercept the negative sum of the edge weights—which is consumed by the ellipsoid and cutting-plane routines of `ellalgo`. The resulting solver handles both linear and nonlinear convex multi-parameter clock skew scheduling; empirically it is reported to be up to an order of magnitude faster than a general linear-programming solver on the linear problem, and several hundred times faster than a general convex-programming solver on the nonlinear problem [11].

4.5 Slack maximization

When no timing violation exists, the natural goal is to make the circuit as robust as possible by *maximizing the minimum slack*. The EVEN method (Section~6) is the simplest instance: maximize β subject to $u_i - u_j \leq \mu_{ij} - \beta$, which is a minimum mean cycle problem. The optional β is the amount of uniform safety margin that can be inserted into every constraint simultaneously. The *minimum balancing* (MB) algorithm [12] realizes this schedule constructively: it finds the most critical cycle, distributes the slack evenly along it, contracts it to a super-vertex, and repeats until a single vertex remains. The contraction order is itself valuable, because it records the *hierarchy of criticality* (Figure~5).

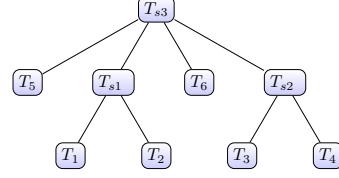


Figure 5: The criticality hierarchy produced by minimum balancing. Each internal node merges a critical cycle; deeper subtrees are less critical.

5 Delay Padding

5.1 Motivation and formulation

When the TCG contains a negative cycle, no clock schedule can repair the circuit; the physical delays themselves must change. *Delay padding* inserts extra delay into selected data paths, usually by swapping a fast cell for a slower one. Its natural formulation adds a nonnegative padding vector p :

$$\text{find } p, u \quad \text{such that} \quad y \leq d + p, \quad Au = y, \quad p \geq 0. \quad (8)$$

If the objective is to minimize the total inserted delay $\sum_e p_e$, then (8) is the *dual* of a standard minimum-cost flow problem, solvable by the network simplex method. This observation is appealing but, in modern designs, *impractical*: delay insertion is realized by cell substitution, so minimizing $\sum p$ does not reflect the true cost, and—more importantly—there may be no valid position to insert delay along some paths.

5.2 The four structural configurations

A better strategy is to decide *where* delay may be inserted *before* deciding *how much* to insert. This is *path relationship analysis* (PRA). For a pair of registers i and j , the maximum-delay (setup) path and the minimum-delay (hold) path interact in one of four ways, and each interaction determines how the TCG must be modified (Figure~6):

1. **Type (a), no insertion.** The two paths coincide or cannot be modified; no delay may be inserted independently, and the TCG is unchanged.
2. **Type (b), independent.** The setup and hold paths are disjoint; delays p_s and p_h may be inserted independently, modelled by auxiliary nodes connected to the original vertices.
3. **Type (c), shared.** The two paths overlap entirely, so a single delay must be shared: $p_s = p_h$.
4. **Type (d), containment.** The hold path is a

sub-path of the setup path; padding upstream affects both, enforcing $p_s \geq p_h$.

5.3 Delay padding as a modified feasibility problem

With the TCG augmented by auxiliary nodes and coupling edges, the question “can the timing be fixed by padding?” again becomes a feasibility problem on a modified graph: an infeasible instance returns a negative cycle, which certifies that padding alone is insufficient and that D_{ij} must be reduced or T_{CP} increased. In the modified graph the padding is a *potential* on the auxiliary edges, so the whole machinery of Section~3 applies unchanged. This “determine the position first, then the value” strategy is both more flexible and more faithful to physical design than the minimize- $\sum p$ flow formulation.

5.4 Discrete padding and its complexity

Formulation (8) treats the padding p as a continuous, nonnegative real vector. In a standard-cell flow this is an idealization: delay is inserted either by swapping a cell for a slower one from a discrete library or by adding whole buffer stages, so the true feasible set of p is *discrete* (and, per path, non-convex), and the exact problem is a mixed-integer linear program (MILP) rather than a network flow. The continuous relaxation studied here should therefore be read as (i) a lower bound on the achievable clock period and (ii) a guide to *which* paths are worth padding. Path relationship analysis narrows the search by pre-selecting physically realizable positions and by coupling the setup and hold paddings ($p_s = p_h$ or $p_s \geq p_h$), but it does not make the values continuous. Closing the gap between a relaxed padding and a discrete, cell-level solution is a separate combinatorial problem; choosing positions *before* values, as above, is what keeps that second problem tractable in practice.

6 Yield-Driven Scheduling under Process Variations

6.1 Timing yield

As technology scales, process variations make delay a random variable. A circuit is functionally correct for a given sample of process parameters if all setup and hold constraints hold; the *timing yield* is the fraction of manufactured instances that are correct. Maximizing yield is fundamentally different from minimizing

a nominal period: it requires reasoning about *distributions* rather than worst-case values [2], [13], [14], [15].

6.2 Primitive solutions and their shortcomings

Three simple heuristics illustrate the difficulty.

1. **Margin pre-allocation.** Reserve a fixed timing margin Δd at both ends of every FSR, i.e., tighten $\underline{w}_{ij} + \Delta d \leq y_{ij} \leq \bar{w}_{ij} - \Delta d$, then optimize the period. This is pessimistic and ignores that different paths have different uncertainty.
2. **Least center error square (LCES).** Place each y_{ij} as close as possible to the center of its FSR by solving a quadratic program. This treats robustness as a distance-to-center problem and is not directly tied to yield.
3. **Incremental slack distribution.** Distribute slack greedily, again ignoring path-delay differences.

All three fail because they do not weight the FSR by the *variance* of the underlying path delays.

6.3 EVEN: uniform slack maximization

The EVEN method assumes that all path delays have equal variance and maximizes a uniform margin β :

$$\max \beta \quad \text{s.t.} \quad u_i - u_j \leq \mu_{ij} - \beta.$$

This is a minimum mean cycle problem, solved by Karp’s or Howard’s algorithm. EVEN then distributes the slack of the most critical cycle evenly, freezes those skews, contracts the cycle to a super-vertex, and repeats (Figure~7). Its weakness is exactly its assumption: a long combinational path usually has larger delay uncertainty than a short one, so even slack distribution is suboptimal for yield.

6.4 PROP: variance-proportional slack

PROP models each gate delay as Gaussian with common variance and a path as the sum of n such delays, so a path delay is $\mathcal{N}(n\mu, n\sigma^2)$ and its standard deviation grows like the square root of the path length. It updates the TCG weights with a parameter α ,

$$\bar{w}_{ij}^{\text{PROP}} = T_{CP} - (D_{ij} + \alpha \sqrt{D_{ij}} \sigma) - T_{\text{setup}}, \quad \underline{w}_{ij}^{\text{PROP}} = T_{\text{hold}} - (d_{ij} - \alpha \sqrt{d_{ij}} \sigma)$$

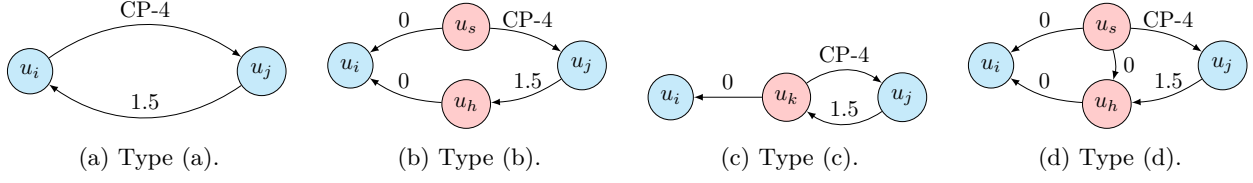
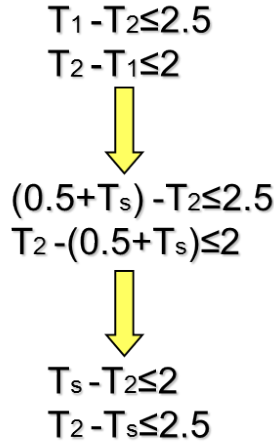
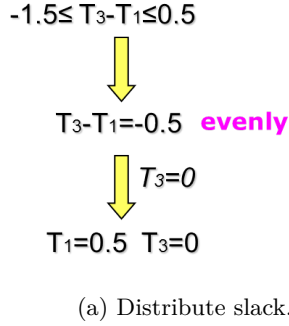
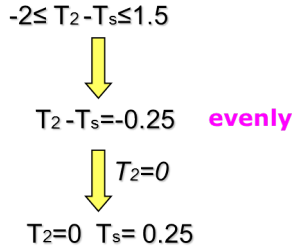


Figure 6: The four structural configurations for delay insertion. Type (a): no feasible insertion; (b): p_s, p_h independent; (c): $p_s = p_h$; (d): $p_s \geq p_h$.



(b) Contract a cycle.



(c) Repeat.

Figure 7: Minimum balancing: identify the most critical cycle, distribute its slack evenly, contract it to a super-vertex, and iterate.

and increases α (using Bellman–Ford as the feasibility oracle) until the graph becomes infeasible. The largest feasible α equalizes the *margins* along the most critical cycle, after which the remaining skews are filled in by EVEN. PROP’s weakness is the assumed common gate distribution and the choice of the square-root weighting, which is a modelling heuristic rather than a derived optimum.

6.5 C-PROP: statistical slack maximization

C-PROP generalizes EVEN to unequal variances by maximizing β subject to

$$u_i - u_j \leq \mu_{ij} - \sigma_{ij}\beta,$$

where μ_{ij} and σ_{ij} are the mean and standard deviation of the edge slack. This is a minimum cost-to-time ratio cycle problem, solved by Howard’s algorithm [16]. Crucially, C-PROP reduces to EVEN when all σ_{ij} are equal, and as a variance tends to zero it allocates only a minimal margin to that constraint while giving the others more—precisely the desired behavior.

6.6 FP-PROP: false-path awareness

A *false path* can never be sensitized and therefore never carries a signal. If false paths are treated as real, non-critical cycles can be mistaken for critical ones, slack is diverted to them, and the truly critical cycles receive too little margin, lowering yield. FP-PROP therefore performs a sensitizable-critical-path search and excludes false paths from the scheduling problem [14].

6.7 Spatial correlation

The statistical formulations above weight each constraint by its own variance σ_{ij} but treat different edges as *independent*. Silicon is not independent: nearby gates and wires share systematic process components, so the delays of physically close edges are positively correlated. Spatial correlation alters both the variance

and the effective criticality of a cycle, and it becomes more pronounced at 3,nm and below. A principled treatment models delay as a Gaussian process over the die. The max-min formulations used here deliberately ignore it, which makes them computationally attractive but not, in general, yield-optimal: they need only the marginal distribution of each constraint and therefore no correlation model—which contemporary STA tools do not report—but this is exactly why the resulting objective is a *surrogate* for, rather than the true, timing yield.

6.8 One parametric family

Table~1 shows that minimum-period scheduling, EVEN, and C-PROP are three members of the single family (6), differing only in the right-hand side $d(\beta)$.

6.9 Gaussian model and linearization

Under a Gaussian delay model the probabilistic constraints can be written explicitly. Maximizing the yield parameter β becomes

$$\begin{aligned} & \text{maximize} && \beta \\ & \text{subject to} && u_i - u_j \leq T_{\text{CP}} - (\mu_{ij}^D + \sigma_{ij}^D \sqrt{2} \operatorname{erf}^{-1}(2\beta - 1)), \\ & && u_j - u_i \leq \mu_{ij}^H + \sigma_{ij}^H \sqrt{2} \operatorname{erf}^{-1}(2(1 - \beta) - 1). \end{aligned}$$

Because erf^{-1} is anti-symmetric and monotone, substituting $\beta' = \sqrt{2} \operatorname{erf}^{-1}(2\beta - 1)$ linearizes the constraints into a minimum cost-to-time ratio problem,

$$u_i - u_j \leq T_{\text{CP}} - \mu_{ij}^D - \sigma_{ij}^D \beta',$$

which can be solved by binary search on β' . The catch is the Gaussian assumption itself, addressed next.

7 Non-Gaussian and Heavy-Tailed Delay Models

7.1 Why the Gaussian model fails

At 65,nm and below, the distribution of a *maximum* path delay is typically asymmetric and heavy-tailed, so the central limit theorem does not rescue the Gaussian assumption. Setup failures occur in slow corners and depend on the upper tail; hold failures occur in fast corners and depend on the lower tail. A model that fits the bulk but misses the tails therefore mis-estimates yield precisely where it matters (Figure~8).



Figure 8: Path-delay distributions are not Gaussian: they are asymmetric and heavy-tailed, so a Gaussian fit captures the bulk but understates the tails that govern setup and hold failures (illustration from the lecture).

7.2 Unimodality and quantile functions

A continuous distribution with mode m is *unimodal* if its CDF is convex for $x < m$ and concave for $x > m$. Normal, log-normal, and log-logistic distributions are unimodal, and their *quantile functions* $z_p = \Phi^{-1}(p)$ are available in closed form:

$$\begin{aligned} \text{Normal:} & \quad \mu + \sigma \sqrt{2} \operatorname{erf}^{-1}(2p - 1), \\ \text{Log-normal:} & \quad \exp(\mu + \sigma \sqrt{2} \operatorname{erf}^{-1}(2p - 1)), \\ \text{Log-logistic:} & \quad \alpha \left(\frac{p}{1-p} \right)^{1/\beta}. \end{aligned}$$

For the log-normal distribution the mode is $\exp(\mu - \sigma^2)$ and the CDF at the mode is $\frac{1}{2}(1 + \operatorname{erf}(-\sigma/\sqrt{2}))$.

7.3 Generalized extreme value distribution

The *generalized extreme value* (GEV) distribution [17] unifies the three extreme-value types and models skewness and tail weight through three parameters: location μ , scale $\sigma > 0$, and shape ξ . Its PDF and CDF are

$$f(x) = \frac{1}{\sigma} t(x)^{\xi+1} e^{-t(x)}, \quad F(x) = e^{-t(x)}, \quad t(x) = \left[1 + \xi \frac{x - \mu}{\sigma} \right]^{-1}$$

The quantile function is

Table 1: Clock skew scheduling as one parametric family. Column headers give the setup (s) and hold (h) edge bounds; all constraints have the form $u_i - u_j \leq d_e(\beta)$.

Problem	objective	$d_{ij}^s(\beta)$	$d_{ij}^h(\beta)$
Min. CP	$\min T_{\text{CP}}$	$T_{\text{CP}} - D_{ij} - T_{\text{setup}}$	$d_{ij} - T_{\text{hold}}$
EVEN	$\max \beta$	$T_{\text{CP}} - D_{ij} - T_{\text{setup}} - \beta$	$d_{ij} - T_{\text{hold}} - \beta$
C-PROP	$\max \beta$	$T_{\text{CP}} - D_{ij} - T_{\text{setup}} - \sigma_{ij}\beta$	$d_{ij} - T_{\text{hold}} - \sigma_{ij}\beta$

$$Q(\beta) = \mu + \frac{\sigma}{\xi} \left((-\ln \beta)^{-\xi} - 1 \right). \quad (9) \quad y \leq d^{(k)}, \quad Au = y, \quad \forall k \in \{1, \dots, K\},$$

Because circuit delays are strictly positive and their maxima are heavy-tailed, the case $\xi \neq 0$ is the relevant one.

7.4 Statistical timing constraints with GEV

With GEV models for the maximum and minimum path delays, the setup and hold constraints become probabilistic,

$$T_{\text{skew}} \leq T_{\text{CP}} - \tilde{D} - T_{\text{setup}}, \quad T_{\text{skew}} \geq T_{\text{hold}} - \tilde{d},$$

and are converted to their deterministic equivalents through the quantile function (9):

$$T_{\text{CP}} - T_{\text{setup}} - T_{\text{skew}} \geq Q_D(\beta), \quad T_{\text{hold}} - T_{\text{skew}} \geq Q_d(1 - \beta).$$

The setup constraint is sensitive to the *upper* tail $Q_D(\beta)$ (slow corners), whereas the hold constraint is sensitive to the *lower* tail $Q_d(1 - \beta)$ (fast corners). Because every CDF is monotone increasing, Q is monotone, so the yield parameter β remains a single scalar and the parametric framework of Section~4 still applies. In general Lawler’s binary search solves the resulting problem; for the special GEV form one may bypass the explicit quantile by searching directly on a transformed parameter, exactly as in the Gaussian linearization of Section~6.

8 Robust Scheduling: Multi-Corner and Multi-Mode

8.1 Meeting constraints in multiple corners

Modern flows must satisfy timing across many process, voltage, and temperature (PVT) corners. With no adjustable delay buffer (ADB), the requirement is

which is equivalent to the single-corner feasibility problem $y \leq \min_k d^{(k)}$. Solving corner by corner is tempting but wrong: a fix applied in one corner can create a violation in another, producing a non-convergent *ping-pong* effect.

8.2 Multi-corner delay padding by dual decomposition

Delay padding across corners cannot be written as a network-flow problem, because the padded delay is shared while the constraints differ per corner. A primal linear program is possible but scales poorly. *Dual decomposition* offers a scalable alternative. Let y_k be the skew vector under corner k and y_{shared} the common target. The coupling $y_k = y_{\text{shared}}$ is relaxed with Lagrange multipliers λ_k , giving

$$\min_{\{y_k\}} \sum_k \lambda_k^T (y_k - y_{\text{shared}}) \quad \text{s.t.} \quad Au = y, \quad \underline{w}^{(k)} \leq y_k \leq \overline{w}^{(k)}.$$

The iterations are simple and parallelizable:

1. solve all K sub-problems independently, one per corner, treating y_{shared} and λ_k as fixed;
2. average the local solutions, $y_{\text{shared}} \leftarrow \frac{1}{K} \sum_{k=1}^K y_k$;
3. update the multipliers by sub-gradient ascent, $\lambda_k \leftarrow \lambda_k + \rho (y_k - y_{\text{shared}})$, with step size $\rho > 0$.

Each sub-problem is a *minimum-cost potential problem*, solvable efficiently; if some sub-problem is infeasible, padding alone cannot fix the design. The process terminates when the local solutions agree; failure to converge is itself a certificate that padding is insufficient. On industrial benchmarks this approach reduces the optimized clock period by roughly six percent relative to a single worst-case corner, and it generalizes the corner-by-corner heuristics of Held et al. [18] and the slack-balancing flow of Wang et al. [19].

8.3 Multi-mode scheduling with adjustable delay buffers

When adjustable delay buffers are added to the clock tree, each mode may have its own arrival times:

$$y^{(m)} \leq d^{(m)}, \quad Au^{(m)} = y^{(m)}, \quad \forall m \in \{1, \dots, M\}.$$

The modes are independent and can be processed in parallel, each by the same negative-cycle strategy. Delay padding across modes is again a shared-variable problem (only the padding p is common), solved by dual decomposition or, if necessary, by linear programming. The hard design questions remaining are how to synthesize a clock tree that realizes a *range* of arrival times per register, how to order the registers by criticality in the presence of modes, and how to size the ADB range minimally.

9 Algorithms

9.1 Overview

Table~2 compares the principal algorithms for the parametric potential problem (6). They share one primitive: the detection and cancellation of negative cycles. The parametric shortest-path algorithm of Young, Tarjan, and Orlin [20] attains the best worst-case bound among them.

9.2 Lawler's binary search

Lawler's method brackets β and halves the interval with a feasibility test (Algorithm~1) at each step. It is easy to implement and robust to nonlinear $d(\beta)$, at the cost of repeated full-graph scans. It converges locally in the sense that each test is global but the bracket shrinks slowly (Figure~9).

9.3 Howard's policy iteration

Howard's algorithm is the method of choice for the single-parameter problem. It maintains one chosen outgoing arc per vertex (a *policy*); the policy graph contains a cycle, whose current ratio gives an improved β ; the potentials are then recomputed with respect to the parameterized weights and the policy is updated. Each iteration produces a strictly better β , and the algorithm returns the optimum together with its critical cycle. Variants that interleave binary-search pivots converge even faster, and hybrid Lawler–Howard methods are common in practice (Figures 10 and 11).

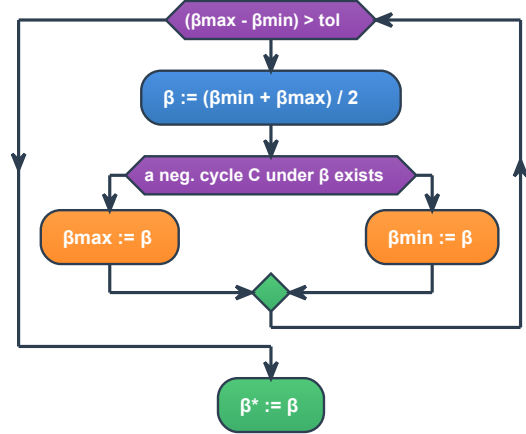


Figure 9: Lawler's binary-search algorithm: bisect the bracket $[\beta_{\min}, \beta_{\max}]$ and test the timing constraint graph for a negative cycle; the upper or lower bound is updated until the tolerance is met.

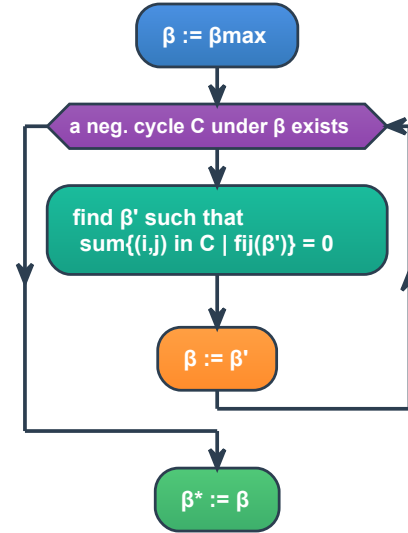


Figure 10: Howard's policy-iteration algorithm: a cycle of the current policy yields an improved β , after which a new policy is computed from the parameterized shortest paths.

Table 2: Algorithms for the parametric potential problem. Here $n = |V|$ and $m = |E|$; ϵ is the binary-search tolerance.

Method	Strategy	Cost / behavior
Bellman–Ford	single feasibility test	$O(nm)$
Karp	minimum mean cycle	$O(nm)$
Lawler	binary search + feasibility test	$O(nm \log(1/\epsilon))$
Howard	policy iteration / cycle cancellation	fast in practice, exact
Young–Tarjan–Orlin	parametric shortest path	$O(nm + n^2 \log n)$



Figure 11: Hybrid and improved variants interleave binary-search pivots with policy iteration, accelerating convergence.

9.4 Karp’s minimum mean cycle

For the unweighted-variance case, the minimum mean cycle admits a closed-form characterization [21], computable in $O(nm)$ time from shortest-path distances with a bounded number of edges. Karp’s algorithm is the canonical subroutine for EVEN and for the minimum-mean-cycle form of any scheduling instance. The broader family of minimum-ratio algorithms is surveyed by Dasdan and Gupta [22].

9.5 From schedule to sequential realizable clock tree

The scheduling algorithms return arrival times, but these must be *realized* by a physical clock tree. Here the network viewpoint again pays off: the minimum balancing algorithm not only produces a schedule but also a contraction tree that encodes the order of criticality (Figure~5). Section~10 discusses how to exploit this structure.

10 Clock-Tree Synthesis and Co-optimization

10.1 Realizing a skew schedule

A clock tree delivering prescribed arrival times is built by *deferred-merge embedding* (DME) with Elmore delay and buffer insertion. Constraining the tree to deliver a *bounded* skew is possible but significantly complicates the algorithm. The practical recommendation is to keep the scheduled skews as a budget: if the schedule is over-optimized, the tree becomes hard to build, and budgeting techniques must be applied. After construction, a detailed timing analysis (not merely Elmore delay) should confirm the arrival times.

Realizing a prescribed skew is not free. Deferred-merge embedding places merging points and inserts buffers to meet target delays, and the buffer count, wirelength, and via count all grow with the *spread* of the required arrival times. Large skews between physically distant registers are the expensive case: they force detours in the clock wiring, raise clock-tree *power*, consume routing resources, and can induce *congestion* that ripples back into signal routing. A schedule that is optimal on the abstract TCG may therefore be unattractive, or even unrealizable, once these costs are counted. This is the central caveat of the approach: the algorithms here optimize a *delay-only* objective, and a production flow must re-optimize with the physical clock tree, or budget the skews so

that the tree stays cheap to build.

10.2 Criticality-ordered topology and placement

A useful heuristic is to make the clock-tree topology follow the criticality hierarchy: registers that belong to the same contracted cycle should share a branch. Two benefits follow. First, a lower branch has smaller skew *variation*, so the most critical registers—placed deepest—enjoy the strongest correlation and the largest cancellation of variation. Second, if the placer also follows the topology—placing registers of the same branch physically together—spatial correlation further reduces skew variation. Since contemporary statistical timing analysis does not report the correlation information needed for a first-principles optimization, aligning topology, placement, and criticality is a valuable, practical surrogate.

10.3 Co-optimization after the tree is built

After clock-tree synthesis the delay estimates are more accurate, raising the question of whether to re-schedule. Some methods relax the constraint to $1.2 u_i - 0.8 u_j \leq w_{ij}$, reflecting that a built tree cannot realize an exact skew; the resulting problem is no longer a pure network flow but remains a linear program. It is also possible to relax the fixed arrival time to an *interval*, which reduces clock-tree wirelength and makes the schedule robust to unavoidable process variation.

11 Discussion and Open Problems

11.1 What the network formulation buys

Framing clock skew scheduling as a parametric potential problem yields three concrete benefits. First, *efficiency*: the combinatorial algorithms are faster than generic linear programming, and the parametric shortest-path methods are faster still. Second, *certificates*: an infeasible instance returns the most critical cycle, which is directly actionable, and a feasible schedule comes with a criticality hierarchy that guides clock-tree synthesis. Third, *extensibility*: delay padding, multi-corner, and multi-mode variants are all obtained by modifying the graph rather than the algorithm.

11.2 Scope and limitations

This article is a survey, not a report of new experiments. Its claims are of two kinds: mathematical statements—the reductions to the parametric potential problem and the negative-cycle criterion—which are exact under the stated model; and practical claims (efficiency, robustness), which are inherited from the cited literature and are not re-validated here.

The model makes four idealizations that a production flow must confront. (i) *Physical realizability*: arrival times are free continuous potentials, whereas a real clock tree incurs power, area, buffer, and congestion costs that grow with skew spread (Section~10). (ii) *Discrete delay*: padding is quantized by cell substitution or buffer insertion, making the exact problem a MILP, of which the continuous network-flow formulation is a relaxation (Section~5). (iii) *Statistical independence*: the yield formulations use per-edge marginals and ignore spatial correlation among nearby paths (Section~6). (iv) *Flatness*: the difference-constraint model assumes one synchronous graph, whereas hierarchical timing abstractions, clock-domain crossings, and dynamic clock gating break that assumption. The framework is thus best understood as the exact *algorithmic core* of a larger physical-optimization problem, to which the surrounding engineering steps add costs and constraints that the core itself does not capture.

11.3 Open problems

Several questions remain open.

- **ADB range.** How to determine the minimum range of adjustable delay buffers that suffices for all modes, and how to synthesize a clock tree realizing a range rather than a value?
- **Criticality under modes.** How to define and compute the order of criticality when multiple modes have different critical cycles?
- **Correlation-aware yield.** The max-min formulations here do not require correlation information, but they are not the true timing yield. Incorporating spatial correlation, as captured by Gaussian processes, remains a challenge.
- **Quantized delay padding.** Delay insertion by cell swapping is inherently quantized; integer or mixed-integer versions of (8) deserve more attention.
- **Non-Gaussian, multivariate yield.** Fitting GEV marginals is only half the story; the joint distribution and its tails govern the true yield.
- **Co-optimization.** Joint placement, clock-tree synthesis, and skew scheduling under a single

objective is still largely unsolved.

- **Physical cost of skew.** The formulations optimize arrival times without charging for clock-tree power, area, buffer count, or routing congestion; a cost-aware objective that prices large, physically distant skews is missing.
- **Hierarchical and gated designs.** Flat difference constraints assume a single synchronous graph; hierarchical timing, clock-domain crossings (CDCs), and dynamic clock gating call for decomposed or interface-constrained formulations.
- **Independent evaluation.** The speedups quoted in this survey are those reported in the cited papers; an apples-to-apples study on modern multi-million-gate benchmarks remains to be done.

12 Conclusion

Clock skew scheduling turns the inevitable, unwanted skew of a clock network into a deliberate design variable. We have shown that the problem has a single mathematical core: the timing constraints are a system of difference constraints, feasibility is the absence of a negative cycle in the timing constraint graph, and the optimization variants—minimum clock period, maximum slack, maximum yield—are parametric shortest-path problems with fast combinatorial solvers. Building on this core, delay padding, multi-corner and multi-mode robustness, dual decomposition, and heavy-tailed GEV delay models extend the framework without changing its algorithmic heart. The critical cycle and the criticality hierarchy returned by these solvers double as actionable design guidance for clock-tree synthesis and placement. As technology scales and process variation dominates, these ideas—useful skew, negative-cycle analysis, and parametric optimization—will remain central to timing closure.

References

- [1] J. P. Fishburn, “Clock skew optimization,” *IEEE Transactions on Computers*, vol. 39, no. 7, pp. 945–951, 1990.
- [2] J. L. Neves and E. G. Friedman, “Optimal clock skew scheduling tolerant to process variations,” in *Proceedings of the 33rd annual design automation conference (DAC)*, 1996, pp. 623–628. doi: [10.1145/240518.240636](https://doi.org/10.1145/240518.240636).
- [3] I. S. Kourtev and E. G. Friedman, *Timing optimization through clock-skew scheduling*. Boston, MA: Springer, 2000.

- [4] W.-S. Luk, “Algorithms for design-for-manufacturability and network optimization: Lecture notes.” Online course notes, <https://luk036.github.io>, 2026.
- [5] R. Bellman, “On a routing problem,” *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [7] E. L. Lawler, *Combinatorial optimization: Networks and matroids*. New York: Holt, Rinehart; Winston, 1976.
- [8] R. A. Howard, *Dynamic programming and markov processes*. Cambridge, MA: MIT Press, 1960.
- [9] L. G. Khachiyan, “A polynomial algorithm in linear programming,” *Soviet Mathematics Doklady*, vol. 20, pp. 191–194, 1979.
- [10] M. Grötschel, L. Lovász, and A. Schrijver, *Geometric algorithms and combinatorial optimization*. Berlin: Springer, 1988.
- [11] X. Zhou, W.-S. Luk, H. Zhou, F. Yang, C. Yan, and X. Zeng, “Multi-parameter clock skew scheduling,” *Integration, the VLSI Journal*, vol. 48, pp. 129–137, 2015, doi: [10.1016/j.vlsi.2014.07.005](https://doi.org/10.1016/j.vlsi.2014.07.005).
- [12] C. Albrecht, B. Korte, J. Schietke, and J. Vygen, “Cycle time and slack optimization for VLSI-chips,” in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 1999, pp. 232–238. doi: [10.1109/ICCAD.1999.810654](https://doi.org/10.1109/ICCAD.1999.810654).
- [13] I. S. Kourtev and E. G. Friedman, “Clock skew scheduling for improved reliability via quadratic programming,” in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 1999, pp. 239–243. doi: [10.1109/ICCAD.1999.810655](https://doi.org/10.1109/ICCAD.1999.810655).
- [14] J.-L. Tsai, D. H. Baik, C. C.-P. Chen, and K. K. Saluja, “Yield-driven, false-path-aware clock skew scheduling,” *IEEE Design & Test of Computers*, vol. 22, no. 3, pp. 214–222, 2005.
- [15] C. Visweswariah, K. Ravindran, K. Kalafala, S. G. Walker, and S. Narayan, “First-order incremental block-based statistical timing analysis,” in *Proceedings of the 41st annual design automation conference (DAC)*, 2004, pp. 331–336.
- [16] X. Wei, Y. Cai, and X. Hong, “Clock skew scheduling under process variations,” in *Proceedings of the 7th international symposium on quality electronic design (ISQED)*, 2006, pp. 237–242.
- [17] A. F. Jenkinson, “The frequency distribution of the annual maximum (or minimum) values of meteorological elements,” *Quarterly Journal of the Royal Meteorological Society*, vol. 81, no. 348, pp. 158–171, 1955.
- [18] S. Held, B. Korte, J. Mäbberg, M. Ringe, and J. Vygen, “Clock scheduling and clock tree construction for high performance ASICs,” in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 2003, pp. 232–239.
- [19] K. Wang, L. Duan, and X. Cheng, “ExtensiveSlackBalance: An approach to make front-end tools aware of clock skew scheduling,” in *Proceedings of the 43rd annual design automation conference (DAC)*, 2006, pp. 951–954. doi: [10.1145/1146909.1147150](https://doi.org/10.1145/1146909.1147150).
- [20] N. E. Young, R. E. Tarjan, and J. B. Orlin, “Faster parametric shortest path and minimum-balance algorithms,” *Networks*, vol. 21, no. 2, pp. 205–221, 1991.
- [21] R. M. Karp, “A characterization of the minimum cycle mean in a digraph,” *Discrete Mathematics*, vol. 23, no. 3, pp. 309–311, 1978.
- [22] A. Dasdan and R. K. Gupta, “Faster maximum and minimum mean cycle algorithms for system-performance analysis,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 17, no. 10, pp. 889–899, 1998.